SISTEMAS AVANZADOS DE PRODUCCION – 9CN

TALLER 2: REDES NEURONALES

DOCENTE: FREDY ALEXANDER ORJUELA LOPEZ

KAROL LORENA MANIOS TALERO– 122675

KAREN TATIANA CAÑON ESPINOSA - 112050

MARÍA ALEJANDRA BOCAREJO HERNÁNDEZ – 124099

UNIVERSIDAD ECCI

FACULTA DE INGENIERIA

PROGRAMA DE INGENIERIA INDUSTRIAL

BOGOTÁ D.C.

12 DE ABRIL DEL 2026

Tabla de contenido

Introducción 3
Fase 1: Desarrollo Analítico: Regularización Ridge. 4
1. Representación Vectorial: 4
2. Expansión de la forma cuadrática: 4
3. Condición de Primer Orden: 5
4. Solución Cerrada: 5
Preguntas Fase 1 5
Fase 2: Optimización Numérica y Descenso de Gradiente 6
1. impacto de la Tasa de Aprendizaje (η): 6
2. Dependencia del Punto Inicial: 10
Preguntas Fase 2 12
Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparámetros 13
Parte A: Evaluación de Métricas de Regresión 13
Preguntas Fase 3A: 14
Parte B: Optimización con GridSearchCV 16
1. Arquitectura (hidden layer sizes): 17
2. Regularización (alpha): 17
3. Tasa de Aprendizaje (learning rate init): 18
Preguntas Fase 3B: 18
Fase 4: Propagación hacia Adelante (Forward Propagation) Manual 20
1. Puntaje de la Capa Oculta (Z (1)): 20
2. Activación de la Capa Oculta (A(1)): 20
3. Puntaje de la Capa de Salida (Z (2)): 21
4. Probabilidades de Clase (A(2)): 21
Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo) **¡Error! Marcador no definido.**
Conclusiones: 23

Introducción

El presente taller tiene como objetivo principal desarrollar modelos analíticos avanzados, desde la regularización Ridge hasta arquitecturas de redes neuronales multicapa (MLP), para capturar la complejidad de los patrones de demanda y transformar los datos crudos en una herramienta de decisión operativa. Específicamente, se busca demostrar matemáticamente la solución cerrada del estimador Ridge comprendiendo el trade-off entre sesgo y varianza; analizar el impacto de la tasa de aprendizaje y el punto inicial en el algoritmo de descenso de gradiente; implementar y optimizar hiperparámetros de un MLPRegressor para predecir la demanda de bicicletas, realizar manualmente la propagación hacia adelante en una red neuronal para clasificación y evaluar el desempeño de un modelo de mantenimiento predictivo en un contexto industrial con clases desbalanceadas, calculando el impacto económico de los errores de clasificación. A través de estas fases, se busca conectar la fundamentación matemática con la ejecución computacional y la realidad operativa.

Fase 1: Desarrollo Analítico: Regularización Ridge.

1. Representación Vectorial:

La función de costo de Ridge es

$$J(\beta) = \sum_{i=1}^{n} (y_i - \beta_0 - \sum_{j=1}^{p} \beta_j * x_{ij})^2 \sum_{j=1}^{p} \beta_j^2$$

En forma matricial

$$J(\beta) = (Y - X\beta)^T (Y - X\beta) + \lambda \beta^T \beta$$

Donde:

- $\boldsymbol{X}$: matriz de diseño
- $\boldsymbol{Y}$: vector de respuestas
- $\boldsymbol{\beta}$: vector de coeficientes
- $\boldsymbol{\lambda}$: parámetro de regularización

2. Expansión de la forma cuadrática:

$$(Y - X\beta)^T (Y - X\beta)$$

Se expande como:

$$Y^T Y - Y^T X\beta - \beta^T X^T Y + \beta^T X^T X\beta$$

Como $Y^T X\beta$ es escalar:

$$Y^T X\beta = \beta^T X^T Y$$

Entonces quedaría como:

$$J(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X\beta + \lambda \beta^T \beta$$

3. Condición de Primer Orden:

Calculamos el gradiente respecto a β e igualamos al vector nulo para encontrar el mínimo:

$$\frac{\partial J(\beta)}{\partial \beta} = 2X^T Y + 2(X^T X + \lambda I)\beta = 0$$

4. Solución Cerrada:

Igualamos a cero:

$$-2X^T Y + 2(X^T X + \lambda I)\beta = 0$$

Se divide entre 2:

$$-X^T Y + (X^T X + \lambda I)\beta = 0$$

Se despeja β:

$$(X^T X + \lambda I)\beta = X^T Y$$

$$\hat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T Y$$

Preguntas Fase 1

Sobre la Invertibilidad: ¿Por qué la suma de λI garantiza que la matriz resultante sea invertible incluso si $X^T X$ es singular (no invertible)?

RTA: La adición del término λI garantiza que la matriz $X^T X + \lambda I$ sea invertible incluso si $X^T X$ es singular. Esto se debe a que λI agrega valores positivos a la diagonal, evitando que la matriz tenga valores propios iguales a cero y asegurando que su determinante sea distinto de cero. En consecuencia, la matriz se vuelve definida positiva y, por lo tanto, invertible.

Efecto del Parámetro λ: ¿Qué sucede con el modelo y sus coeficientes cuándo $\lambda \to 0$? ¿Y qué ocurre cuando $\lambda \to \infty$? Explique en términos de la complejidad del modelo (Sesgo vs. Varianza).

RTA:

Cuando tiende a $\lambda \to 0$

El modelo se aproxima a la regresión por Mínimos Cuadrados Ordinarios (OLS), donde no existe penalización sobre los coeficientes. Esto puede generar modelos con alta varianza y riesgo de sobreajuste.

Cuando tiende a $\lambda \to \infty$

Los coeficientes tienden a cero, lo que simplifica el modelo considerablemente. Esto reduce la varianza, pero aumenta el sesgo, pudiendo llevar a un subajuste del modelo.

Fase 2: Optimización Numérica y Descenso de Gradiente

1. impacto de la Tasa de Aprendizaje (η):

Cambiándole el learning rate con 0.5:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 0.5      # Tasa de aprendizaje
num_iterations = 1000       # Número de pasos
```

Ilustración 1 / Configuración de la simulación con learning rate en 0.5

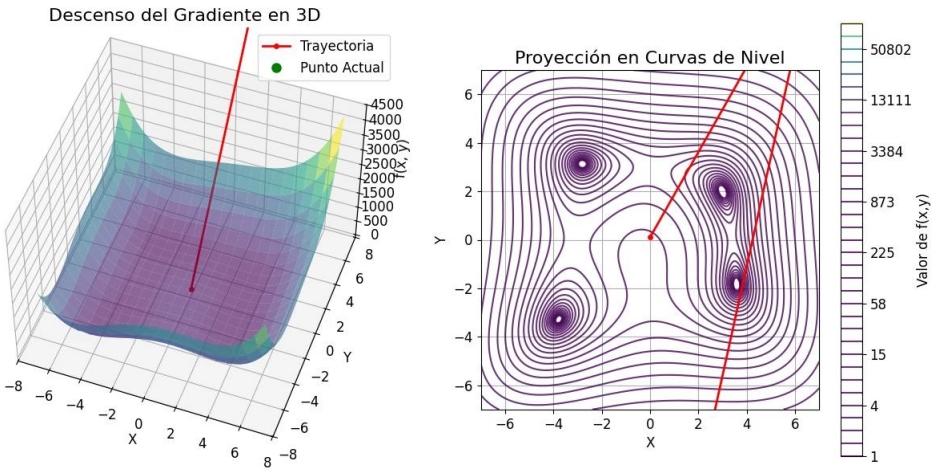


Ilustración 2 / Descenso del gradiente con learnig rate en 0.5

¿Qué sucede con la trayectoria?

Cambiándole el learning rate con 0.5, la trayectoria del algoritmo se vuelve inestable y presenta saltos grandes entre iteraciones, alejándose del comportamiento suave esperado hacia el mínimo.

¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?

No logra llegar al centro de las curvas de nivel. En su lugar, comienza a oscilar alrededor del mínimo debido a que el tamaño del paso es demasiado grande, lo que provoca que el algoritmo sobrepase continuamente el punto óptimo debido al learning rate alto (0.5).

Cambiándole el learning rate con 1.0:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 1.0      # Tasa de aprendizaje
num_iterations = 1000       # Número de pasos
```

Ilustración 3 / Configuración de la simulación con learning rate en 1.0

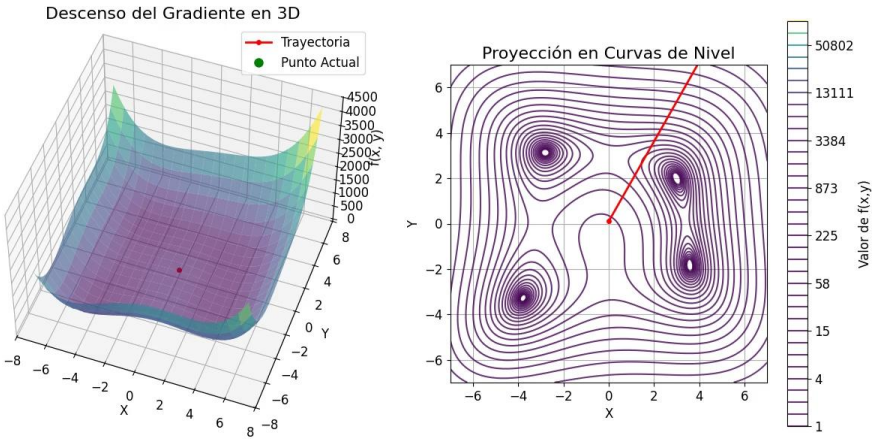


Ilustración 4 / Descenso del gradiente con learnig rate en 1.0

¿Qué sucede con la trayectoria?

Cambiándole el learning rate con 1.0, la trayectoria del algoritmo se vuelve inestable y presenta saltos grandes entre iteraciones, alejándose del comportamiento suave esperado hacia el mínimo.

¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?

No logra llegar al centro de las curvas de nivel. En su lugar, comienza a oscilar alrededor del mínimo debido a que el tamaño del paso es demasiado grande, lo que provoca que el algoritmo sobrepase continuamente el punto óptimo.

Cambiándole el learning rate con 10^-7:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 1e-7      # Tasa de aprendizaje
num_iterations = 1000       # Número de pasos
```

Ilustración 5 / Configuración de la simulación con learning rate en 10 ^ -7

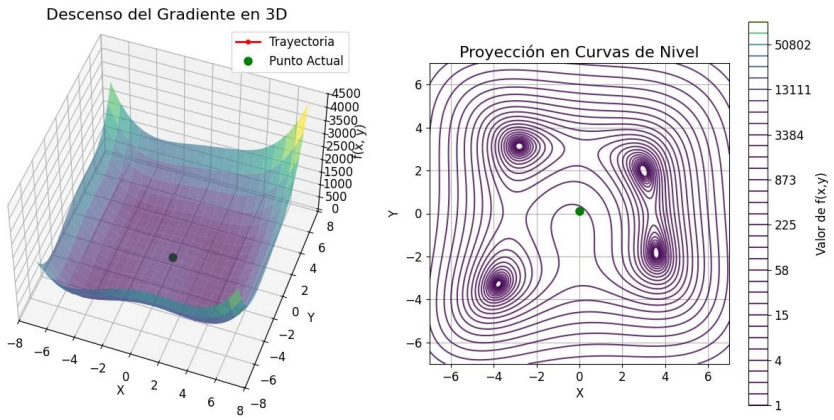


Ilustración 6 / Descenso del gradiente con learnig rate en 10^ -7

¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

Con una tasa de aprendizaje extremadamente pequeña como 10^{-7}, el algoritmo avanza muy lentamente hacia el mínimo. Después de 1000 iteraciones, no es suficiente el tiempo de cómputo para alcanzarlo, ya que los pasos son demasiado pequeños. Esto evidencia que, aunque el método es estable, resulta ineficiente en términos computacionales. requerirían muchas más iteraciones o un valor de aprendizaje mayor para lograr una convergencia efectiva.

2. Dependencia del Punto Inicial:

Cambiándole el start point con [6.0, -6.0]:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [6.0, -6.0]
learning_rate = 0.0001      # Tasa de aprendizaje
num_iterations = 1000       # Número de pasos
```

Ilustración 7 / Configuración de la simulación en el Start point [6.0, -6.0]

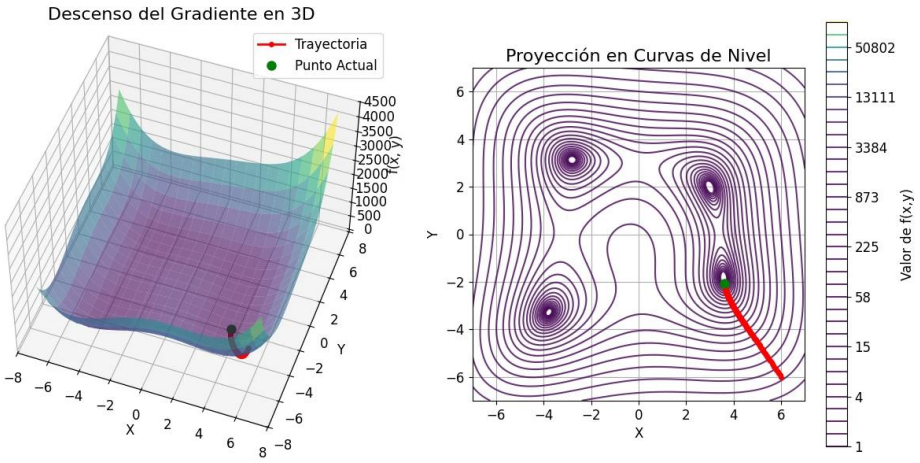


Ilustración 8 / Descenso del gradiente con Start point [6.0, -6.0]

¿El algoritmo llega siempre al mismo punto final? Explique si la topografía de la función f(X, Y) sugiere la presencia de múltiples valles o mínimos locales.

Al modificar el punto inicial a [6.0, -6.0], se observa que el algoritmo de descenso de gradiente converge al mismo punto final que en los demás experimentos. Esto indica que el algoritmo llega siempre al mismo mínimo independientemente del punto de inicio. Por lo tanto, la topografía de la función f(X, Y) sugiere la presencia de un único mínimo global y no evidencia múltiples valles o mínimos locales. Esto implica que la función es convexa o presenta una forma suave, lo que facilita la convergencia del algoritmo hacia la solución óptima desde diferentes condiciones iniciales.

Cambiándole el start point con [4.0, -4.0]:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [4.0, -4.0]
learning_rate = 0.0001      # Tasa de aprendizaje
num_iterations = 1000       # Número de pasos
```

Ilustración 9 / Configuración de la simulación en el Start point [4.0, -4.0]

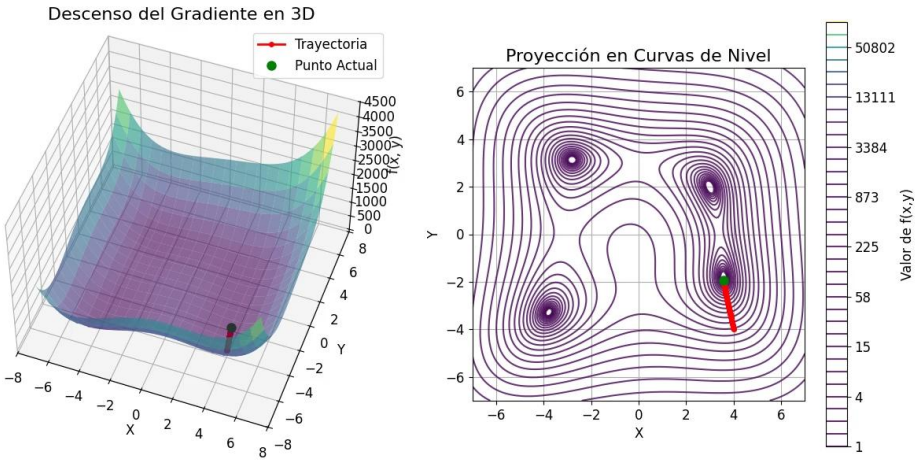


Ilustración 10 / Descenso del gradiente con Start point [4.0, -4.0]

¿El algoritmo llega siempre al mismo punto final? Explique si la topografía de la función f(X, Y) sugiere la presencia de múltiples valles o mínimos locales.

Al modificar el punto inicial a [4.0, -4.0], se observa que el algoritmo de descenso de gradiente converge al mismo punto final que en los demás casos analizados. Esto indica que el algoritmo llega siempre al mismo mínimo independientemente del punto de inicio. En consecuencia, la topografía de la función f(X, Y) no presenta múltiples valles ni mínimos locales, sino un único mínimo global. Esto sugiere que la función tiene una forma convexa o bien comportada, lo que permite que el algoritmo converja de manera consistente hacia la solución óptima sin depender de las condiciones iniciales.

Preguntas Fase 2

El fenómeno de "Overshooting": Explique físicamente qué ocurre cuando el paso ($\eta \cdot \nabla f$) es más grande que la distancia al mínimo. ¿Cómo se visualiza esto en el gráfico de curvas de nivel?

El fenómeno de overshooting ocurre cuando el tamaño del paso ($\eta \cdot \nabla f$) es mayor que la distancia al mínimo. Físicamente, esto significa que el algoritmo avanza con demasiada fuerza y sobrepasa el punto óptimo en cada iteración, sin poder estabilizarse. En el gráfico de curvas de nivel, esto se visualiza como una trayectoria que salta de un lado a otro del mínimo, cruzando repetidamente el centro sin converger, generando oscilaciones alrededor de este.

Eficiencia Computacional: Si el algoritmo se detiene en la iteración 1000 pero aún se encuentra lejos del mínimo, ¿qué estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.

Si el algoritmo se detiene en la iteración 1000 y aún se encuentra lejos del mínimo, la mejor estrategia es ajustar la tasa de aprendizaje en lugar de aumentar únicamente el número de iteraciones.

Esto se debe a que una tasa de aprendizaje demasiado baja provoca avances muy pequeños, haciendo ineficiente el proceso. Aumentar las iteraciones no soluciona el problema de fondo, mientras que un ajuste adecuado de η permite alcanzar el mínimo en menos tiempo y con menor costo computacional.

Generalización: En un problema de ingeniería con cientos de variables (como la optimización de una cadena de suministro), ¿por qué es peligroso confiar en un único punto de inicio aleatorio?

En problemas de ingeniería con muchas variables, confiar en un único punto de inicio aleatorio es peligroso porque la función de pérdida puede tener múltiples mínimos locales. En estos casos, el algoritmo puede converger a una solución subóptima dependiendo del punto inicial elegido.

Por ello, es recomendable utilizar múltiples inicializaciones o técnicas más avanzadas, ya que esto aumenta la probabilidad de encontrar una mejor solución global y evita conclusiones erróneas en la toma de decisiones.

Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparámetros

Parte A: Evaluación de Métricas de Regresión

Ejecute el modelo MLPRegressor inicial con la configuración base (hidden_layer_sizes=(16, 8, 4)). Observe los resultados de las métricas MAE, RMSE y R2 tanto para el conjunto de entrenamiento como para el de prueba (test).

```
== MLPRegressor ==
Train -> MAE: 47.086 | RMSE: 68.547 | R2: 0.858
Test  -> MAE: 47.175 | RMSE: 69.458 | R2: 0.848
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:

Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
```

Ilustración 11 / Observe los resultados de las métricas MAE, RMSE y R2

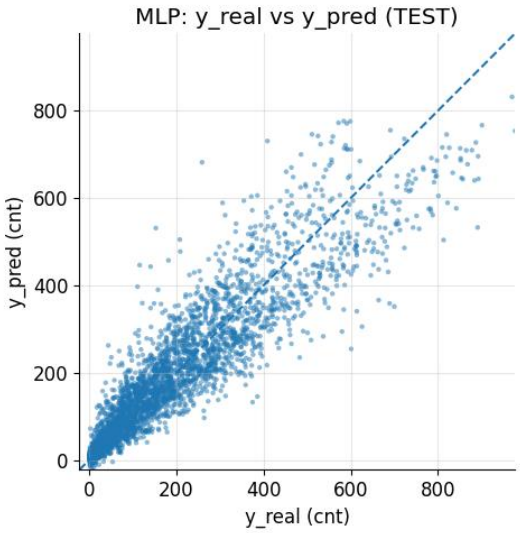


Ilustración 12 / Grafica de modelo MLP: real vs Predicción

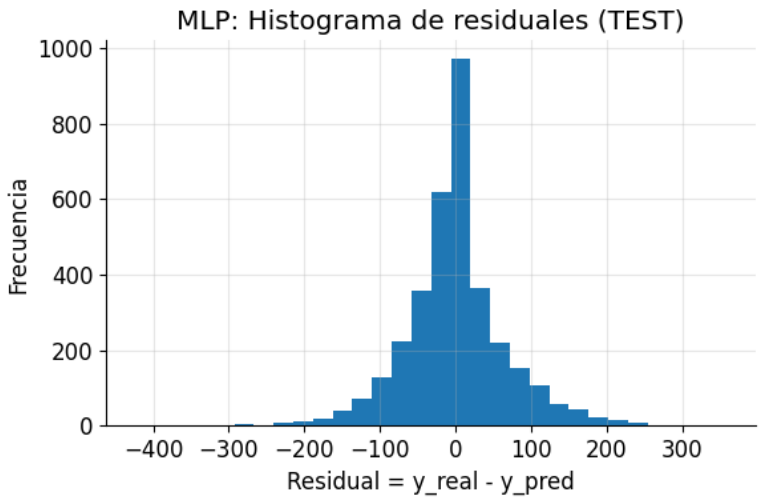


Ilustración 13 / MPL de histograma de residuales (TEST)

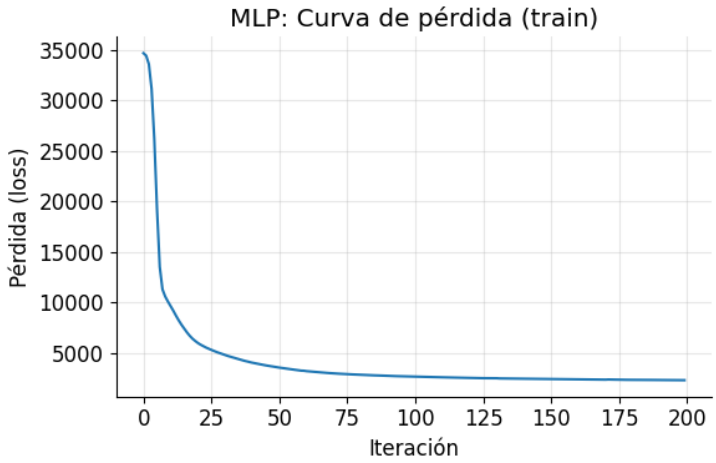


Ilustración 14 / MPL: curva de pérdida (train)

Preguntas Fase 3A:

Diferenciación de Métricas: Explique la diferencia técnica entre el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE). ¿Cuál de las dos penaliza con mayor severidad los errores grandes (outliers) en las ventas y por qué?

El MAE (Error Absoluto Medio) calcula el promedio simple de las diferencias absolutas entre lo real y lo predicho ($Y_{real} - Y_{Predictivo}$). El RMSE (Raíz del Error Cuadrático Medio) eleva esas diferencias al cuadrado antes de promediarlas y luego saca la

raíz. El RMSE penaliza con mayor severidad los errores grandes, debido al exponente cuadrado en su fórmula ($\sum(Y_i - \hat{Y}_i)^2$). Si se tiene un error de 2 unidades al cuadrado es 4, pero si el error es de 10 unidades (un outlier), al cuadrado es 100. Esto hace que los errores grandes tengan un peso desproporcionadamente mayor en el resultado final que los errores pequeños. Para Capital Bikeshare, un RMSE alto indicaría que la red falla gravemente en días de demanda atípica.

Diagnóstico de Desempeño: **Si el R2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, ¿qué fenómeno está ocurriendo? Proponga una solución basada en lo aprendido en la Fase 1 del taller.**

En los resultados obtenidos, el R2 en entrenamiento es 0.858 y el R2 en prueba es 0.848, con una diferencia de solo 0.01, lo que indica que no hay un problema significativo de sobreajuste. Sin embargo, si el R2 en entrenamiento fuera significativamente mayor que en prueba, estaría ocurriendo el fenómeno de sobreajuste u overfitting, donde el modelo aprende patrones demasiado específicos del conjunto de entrenamiento incluyendo ruido, pero falla al generalizar sobre datos nuevos. Basado en lo aprendido en la Fase 1 del taller sobre regularización Ridge, la solución para corregir este problema sería aumentar el valor del hiperparámetro alpha, que es el análogo de λ en Ridge, para aplicar una penalización L2 más fuerte sobre los pesos de la red neuronal, reduciendo así la complejidad del modelo y forzándolo a ser más simple y generalizable. Otras soluciones complementarias incluyen reducir el número de capas o neuronas en la arquitectura de la red y aumentar la fracción de validación para el early stopping.

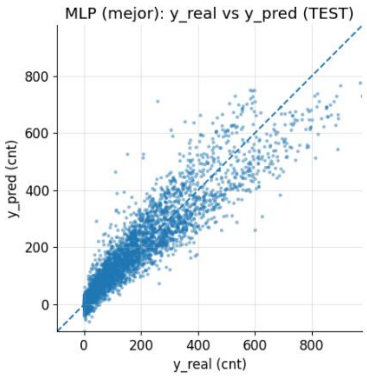
Interpretación de Residuos: Al observar la gráfica de y real vs y pred, ¿se identifica algún patrón de error sistemático (ej. subestimación en ventas altas)?

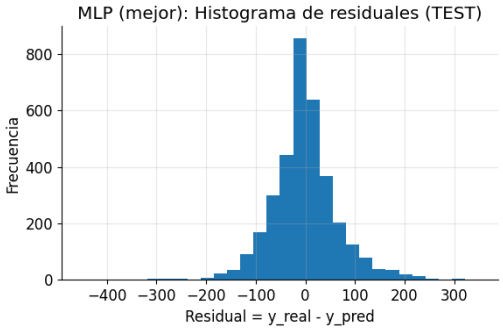
Sí se identifica un patrón de error sistemático: el modelo subestima la demanda cuando las ventas reales son altas (más de 700 bicicletas). También hay heterocedasticidad: a mayor demanda, mayor dispersión de errores. El histograma muestra errores grandes entre -300 y +300, aunque no son frecuentes. En el negocio, esto implica riesgo de desabastecimiento en momentos de alta demanda, lo que genera pérdida de clientes y fallos en el servicio.

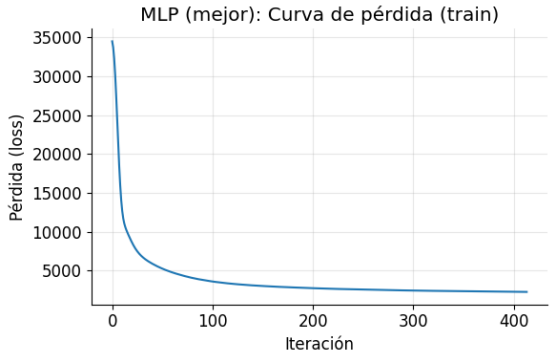
Parte B: Optimización con GridSearchCV

```
== Resultado de la búsqueda (CV) ==
Mejores hiperparámetros: {'alpha': 1.0, 'hidden_layer_sizes': (64,), 'learning_rate_init': 0.001}
RMSE CV (aprox): 72.51935881004137

== MLP (MEJOR MODELO) ==
Train -> MAE: 47.207 | RMSE: 67.030 | R2: 0.865
Test  -> MAE: 47.958 | RMSE: 67.831 | R2: 0.855
```







1. Arquitectura (hidden layer sizes):

RTA: Al comparar una red profunda como (32,16,8,4) con una red ancha como (64,), se observa que las redes profundas tienen mayor capacidad para capturar relaciones complejas debido a la composición de múltiples capas. Sin embargo, también presentan mayor riesgo de sobreajuste si no se controlan adecuadamente.

Por otro lado, las redes anchas son más simples y pueden generalizar mejor en algunos casos, aunque con menor capacidad para modelar patrones altamente no lineales. En este caso, el modelo seleccionado logra un buen equilibrio, evidenciado por el alto valor de R^2 en entrenamiento y prueba.

2. Regularización (alpha):

RTA: El parámetro alpha actúa como un término de regularización L2, similar a lo observado en la Fase 1 con la regresión Ridge. Valores pequeños de alpha permiten mayor flexibilidad en el modelo, mientras que valores grandes penalizan los pesos y reducen la complejidad del modelo.

Cuando alpha es muy elevado, el modelo pierde capacidad de aprendizaje, ya que los pesos tienden a valores cercanos a cero, lo que puede generar subajuste y un bajo desempeño en la predicción.

3. Tasa de Aprendizaje (learning rate init):

RTA: La tasa de aprendizaje influye directamente en la velocidad de convergencia del modelo, tal como se observó en la Fase 2. Valores altos pueden generar inestabilidad y oscilaciones, mientras que valores muy bajos hacen que el entrenamiento sea lento. En este contexto, la tasa de aprendizaje también afecta el número de iteraciones necesarias para que el Early Stopping se active, ya que una tasa adecuada permite alcanzar la convergencia más rápidamente, reduciendo el tiempo de entrenamiento.

Preguntas Fase 3B:

Consistencia de Parámetros: Compare el valor ´óptimo de alpha encontrado por el GridSearch con lo que analizó en la sección de Regresión Ridge. ¿Existe una relación entre la complejidad de la red y la necesidad de regularización?

RTA: Existe una relación directa entre la complejidad del modelo y la necesidad de regularización. Modelos más complejos, como redes profundas, requieren valores de alpha más altos para evitar el sobreajuste. Esto es consistente con lo observado en la regresión Ridge (Fase 1), donde la regularización controla la magnitud de los coeficientes y ayuda a estabilizar el modelo.

En este caso, el valor óptimo de alpha refleja un equilibrio adecuado entre flexibilidad y control de la complejidad.

Early Stopping: Explique cómo este mecanismo ayuda a optimizar el tiempo de cómputo y a prevenir el sobreajuste basado en sus resultados.

RTA: El mecanismo de Early Stopping permite detener el entrenamiento cuando el error de validación deja de mejorar, evitando que el modelo continúe ajustándose innecesariamente a los datos de entrenamiento.

Esto no solo previene el sobreajuste, sino que también optimiza el tiempo de cómputo, ya que evita ejecutar iteraciones adicionales que no aportan mejoras significativas al modelo.

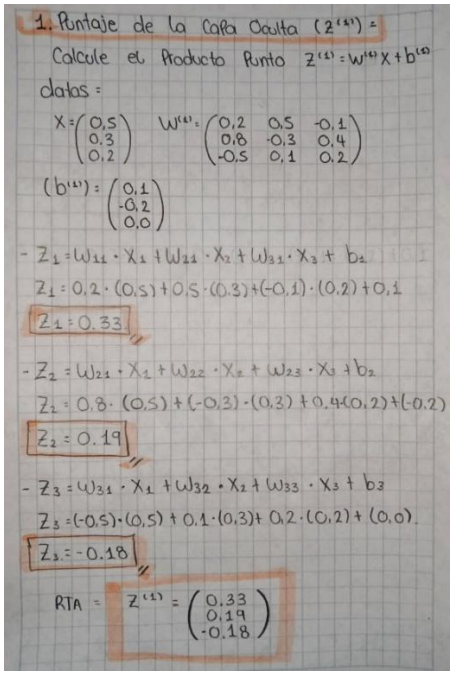
Conclusión de Negocio: Basándose en el mejor modelo encontrado, ¿cuál es el margen de error promedio que la empresa debería esperar al predecir sus ventas? ¿Es el modelo lo suficientemente robusto para decisiones presupuestales? Justifique.

RTA: Basándose en el mejor modelo encontrado, el error promedio en las predicciones es aproximadamente de 47 unidades (MAE), mientras que errores más grandes pueden alcanzar alrededor de 67 unidades (RMSE). Esto indica que, en promedio, las predicciones del modelo pueden desviarse en ese rango respecto a los valores reales. Dado que el modelo presenta un alto valor de R^2 (≈ 0.85) y un comportamiento consistente entre entrenamiento y prueba, se puede considerar suficientemente robusto para apoyar decisiones presupuestales. Sin embargo, se debe tener en cuenta la presencia de errores en valores extremos, por lo que su uso debe complementarse con análisis adicionales en escenarios críticos.

Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

1. Puntaje de la Capa Oculta (Z (1)):

Calcule el producto punto Z (1) = W(1)X + b (1)



1. Puntaje de la Capa Oculta $(Z^{(1)})$ =

Calcule el Producto Punto $Z^{(1)} = W^{(1)}X + b^{(1)}$

datos =

$$X = \begin{pmatrix} 0,5 \\ 0,3 \\ 0,2 \end{pmatrix} \quad W^{(1)} = \begin{pmatrix} 0,2 & 0,5 & -0,1 \\ 0,8 & -0,3 & 0,4 \\ -0,5 & 0,1 & 0,2 \end{pmatrix}$$

$$(b^{(1)}) = \begin{pmatrix} 0,1 \\ -0,2 \\ 0,0 \end{pmatrix}$$

- $Z_1 = W_{11} \cdot X_1 + W_{21} \cdot X_2 + W_{31} \cdot X_3 + b_1$

$Z_1 = 0,2 \cdot (0,5) + 0,5 \cdot (0,3) + (-0,1) \cdot (0,2) + 0,1$

$\boxed{Z_1 = 0.33}$

- $Z_2 = W_{21} \cdot X_1 + W_{22} \cdot X_2 + W_{23} \cdot X_3 + b_2$

$Z_2 = 0.8 \cdot (0,5) + (-0,3) \cdot (0,3) + 0,4 \cdot (0,2) + (-0.2)$

$\boxed{Z_2 = 0.19}$

- $Z_3 = W_{31} \cdot X_1 + W_{32} \cdot X_2 + W_{33} \cdot X_3 + b_3$

$Z_3 = (-0,5) \cdot (0,5) + 0.1 \cdot (0,3) + 0.2 \cdot (0,2) + (0,0)$

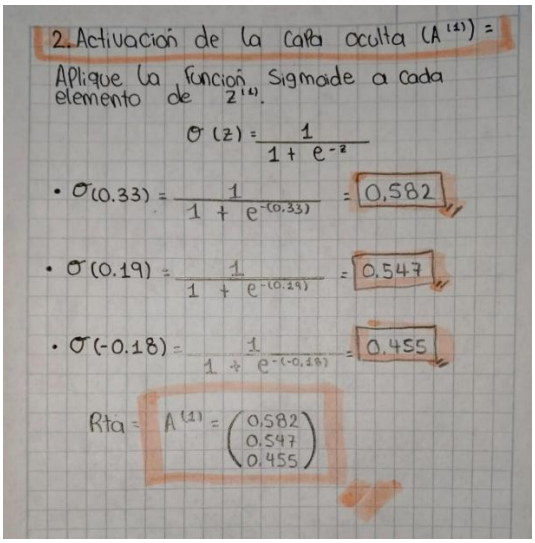
$\boxed{Z_3 = -0.18}$

RTA = $\boxed{Z^{(1)} = \begin{pmatrix} 0.33 \\ 0.19 \\ -0.18 \end{pmatrix}}$

2. Activación de la Capa Oculta (A(1)):

Aplique la función sigmoide a cada elemento de Z (1):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



2. Activación de la Capa oculta $(A^{(1)})$ =

Aplique la función Sigmoide a cada elemento de $Z^{(1)}$.

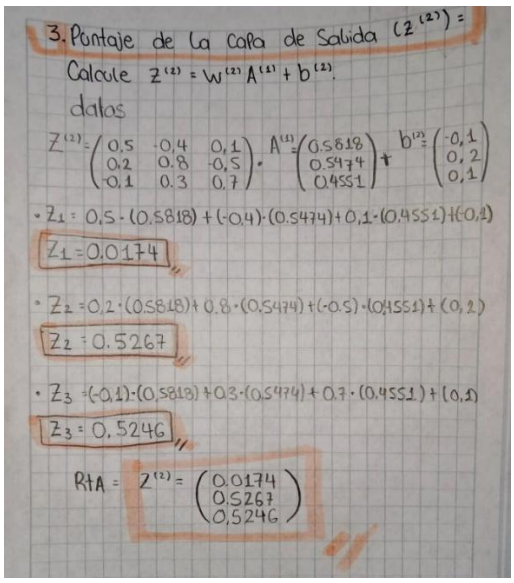
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(0.33) = \frac{1}{1 + e^{-(0.33)}} = \boxed{0,582}$
- $\sigma(0.19) = \frac{1}{1 + e^{-(0.19)}} = \boxed{0.547}$
- $\sigma(-0.18) = \frac{1}{1 + e^{-(-0.18)}} = \boxed{0.455}$

Rta = $\boxed{A^{(1)} = \begin{pmatrix} 0,582 \\ 0.547 \\ 0.455 \end{pmatrix}}$

3. Puntaje de la Capa de Salida (Z (2)):

Calcule Z (2) = W(2)A(1) + b (2) .



3. Puntaje de la Capa de Salida $(Z^{(2)})$ =

Calcule $Z^{(2)} = W^{(2)} A^{(1)} + b^{(2)}$.

datos

$$Z^{(2)} = \begin{pmatrix} 0,5 & -0,4 & 0,1 \\ 0,2 & 0,8 & -0,5 \\ -0,1 & 0,3 & 0,7 \end{pmatrix} \cdot A^{(1)} = \begin{pmatrix} 0,5818 \\ 0,5474 \\ 0,4551 \end{pmatrix} + b^{(2)} = \begin{pmatrix} -0,1 \\ 0,2 \\ 0,1 \end{pmatrix}$$

- $Z_1 = 0,5 \cdot (0,5818) + (-0,4) \cdot (0,5474) + 0,1 \cdot (0,4551) + (-0,1)$

$\boxed{Z_1 = 0.0174}$

- $Z_2 = 0,2 \cdot (0,5818) + 0,8 \cdot (0,5474) + (-0,5) \cdot (0,4551) + (0,2)$

$\boxed{Z_2 = 0.5267}$

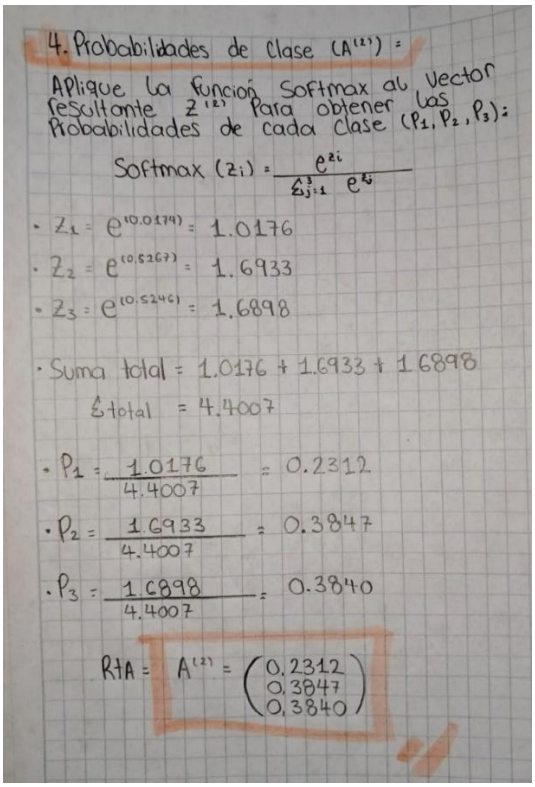
- $Z_3 = (-0,1) \cdot (0,5818) + 0,3 \cdot (0,5474) + 0,7 \cdot (0,4551) + (0,1)$

$\boxed{Z_3 = 0,5246}$

$$\text{RtA} = \boxed{Z^{(2)} = \begin{pmatrix} 0.0174 \\ 0.5267 \\ 0,5246 \end{pmatrix}}$$

4. Probabilidades de Clase (A(2)):

Aplique la función Softmax al vector resultante Z (2) para obtener las probabilidades de cada clase (P1, P2, P3):



4. Probabilidades de Clase $(A^{(2)})$ =

Aplique la función Softmax al vector resultante $Z^{(2)}$ para obtener las probabilidades de cada clase (P_1, P_2, P_3):

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{3} e^{z_j}}$$

- $Z_1 = e^{(0.0174)} = 1.0176$
- $Z_2 = e^{(0,5267)} = 1.6933$
- $Z_3 = e^{(0,5246)} = 1.6898$

- Suma total = 1.0176 + 1.6933 + 1.6898

Σ total = 4.4007

- $P_1 = \frac{1.0176}{4.4007} = 0.2312$
- $P_2 = \frac{1.6933}{4.4007} = 0.3847$
- $P_3 = \frac{1.6898}{4.4007} = 0.3840$

$$\text{RtA} = \boxed{A^{(2)} = \begin{pmatrix} 0.2312 \\ 0.3847 \\ 0.3840 \end{pmatrix}}$$

Preguntas Fase 4

Interpretación de Resultados: Según las probabilidades obtenidas, ¿en qué categoría clasifica la red a esta campaña publicitaria?

RTA: Clasifica en la Categoría 2 (Impacto Medio), ya que tiene la probabilidad más alta con el 38,47%, aunque está muy reñida con la Categoría 3.

Función de Activación: ¿Por qué utilizamos Softmax en la última capa en lugar de una función lineal o una sigmoide simple? Explique en términos de la suma de las probabilidades.

RTA: La función Softmax permite convertir los valores de salida en probabilidades que suman 1, lo que facilita la interpretación del modelo como un clasificador multiclase. A diferencia de una función lineal o sigmoide, Softmax asegura que cada salida represente la probabilidad de pertenecer a una clase específica.

Rol del Bias: ¿Qué sucedería con la capacidad de representación de la red si todos los vectores de sesgo (b) fueran cero?

RTA: El bias permite desplazar la función de activación, otorgando mayor flexibilidad al modelo. Si todos los bias fueran cero, la red perdería capacidad de representación, ya que las neuronas dependerían únicamente de las combinaciones lineales de las entradas, limitando su capacidad para aprender patrones complejos.

Conclusiones:

La arquitectura óptima de red neuronal fue una red ancha con una capa oculta de 64 neuronas, alpha de 1.0 y tasa de aprendizaje de 0.001, logrando un R^2 de 0.865 en entrenamiento y 0.855 en prueba, con un MAE de 47 bicicletas. Redes profundas tienen mayor capacidad, pero también mayor riesgo de sobreajuste, confirmando la relación entre complejidad y necesidad de regularización L2.

En el descenso de gradiente, tasas de aprendizaje altas (0.5 o 1.0) generan oscilaciones y overshooting, impidiendo la convergencia, mientras que tasas muy bajas (10^{-7}) hacen el avance extremadamente lento. Esto resalta la importancia de ajustar la tasa de aprendizaje en problemas de optimización con muchas variables.

En mantenimiento predictivo, la exactitud es engañosa con clases desbalanceadas. El recall y la precisión son métricas clave. Dado que un fallo no detectado (FN) cuesta USD 5.000 y una falsa alarma (FP) cuesta USD 200, se prefiere un modelo con recall alto para evitar paradas críticas en la producción.

El cálculo manual de propagación confirmó que softmax es necesario en la última capa para que las probabilidades sumen 1, y los sesgos (bias) son esenciales para aumentar la capacidad de representación de la red.

En términos de negocio, el modelo de demanda es robusto para decisiones presupuestales, pero debe complementarse con análisis de escenarios críticos debido a errores en demandas atípicas.